

Hito 4 MVP-1 funcional

Jorge Beneyto
Salet Díaz
Kaxi Liu

Hito 4 Diseño y planificación

Hito 3 MVP-1 funcional	1
Hito 2 Diseño y planificación	2
1.Objetivos SMART	3
2.Alcance mínimo (MVP) y ampliaciones.	4
Alcance Mínimo	4
Ampliaciones	4
3.Riesgos y cómo se mitigarán.	4
Riesgos de desarrollo y gestión:	4
Riesgos técnicos:	5
4. Arquitectura de alto nivel	6
5.Casos de uso UML	6
6. Diagrama de Entidad Relación	8
6.1. Entidades principales:	8
7. Plan de pruebas inicial	9
8.Mockups navegables de todas las pantallas	11
HITO 3	21
1. Resumen del MVP-1 y alcance implementado	21
1.1. Cambios que se hizo en el diseño E-R en el Hito 2:	21
2. Funcionalidades del MVP-1 (Casos de Uso Implementados)	22
3. Arquitectura Implementada y Entorno de Ejecución	23
4.Base de datos y operaciones CRUD implementadas	23
5. Plan de pruebas ejecutado (con resultado)	24
6. Repositorio y control de versiones	24

1.Objetivos SMART



Objetivo 1 (Interfaz): Diseñar un prototipo de alta fidelidad y navegable en Figma que incluya las cinco pantallas principales del flujo de usuario (Inicio, Ranking, Tienda del Pingüino, Página del Pingüino y Perfil), asegurando la coherencia visual y la navegabilidad entre todas ellas, antes del 20 de noviembre de 2025.

(Actualizado Hito 4)

Objetivo 2 (Técnico): Implementar el backend utilizando los servicios de Firebase (Auth, Firestore y Storage) para gestionar la autenticación y la base de datos en la nube (NoSQL). La aplicación móvil nativa para Android será desarrollada en Kotlin utilizando Jetpack Compose y una arquitectura MVVM + Repository. Además, la persistencia de datos local (para su uso offline) se realizará con Room Database (SQLite).

Objetivo 3 (Funcional): Desarrollar una aplicación Android de gestión de hábitos que permita a los usuarios registrarse, iniciar sesión, agregar, cumplir y eliminar hábitos, antes del enero de 2026.

Objetivo 4 (Calidad): Definir y ejecutar un plan de pruebas que valide el login, el CRUD de hábitos y el manejo de errores, asegurando que al menos el 80% de los casos de prueba definidos pasen correctamente, con los resultados documentados antes del febrero de 2026.

Objetivo 5 (Transversal): Digitalizar el proceso de seguimiento de hábitos, que tradicionalmente se realiza en papel o con recordatorios mentales, mediante una

aplicación móvil accesible, fomentando la digitalización en la vida cotidiana de los usuarios

2.Alcance mínimo (MVP) y ampliaciones.

Alcance Mínimo

- Registro e inicio de sesión seguro.
- CRUD completo de hábitos: Crear, visualizar en lista, marcar como cumplido y eliminar hábitos.
- Historial visual del progreso de cada hábito.

Ampliaciones

- Gamificación: Sistema de recompensas con monedas, tienda para personalizar al pingüino y ranking semanal.
 - Notificaciones push para recordatorios de hábitos.
 - Funcionalidades sociales, como un chat entre amigos.
- (HITO 4)
- Degradación y sed del pinguino: El sistema calcula los días de inactividad basándose en marcas de tiempo, empeorando el nivel de sed de la mascota un 10% por cada día que el usuario no abre la app.

3.Riesgos y cómo se mitigarán.

Riesgos de desarrollo y gestión:

- Falta de tiempo efectivo por carga de otros módulos (FEs): riesgo alto.
 - *Mitigación:* Planificar el tiempo que podemos utilizar debidamente.
- Alcance excesivo o mala estimación de complejidad: riesgo alto.
 - *Mitigación:* Trabajar principalmente en la versión mínima, y ya una vez terminada, si tenemos tiempo, desarrollar más la app.
- Pérdida de seguimiento o mala comunicación entre miembros: riesgo bajo.
 - *Mitigación:* Usar [Trello](#)/GitHub Projects, poder comunicarnos constantemente mediante un grupo de whatsapp y no esperar a las sesiones en clase para comunicar los posibles problemas, sino poder comunicarlos de forma inmediata
- Abandono de un integrante del equipo: riesgo bajo.
 - *Mitigación:* Favorecer la comunicación y bienestar de los compañeros, rotar de actividades para no “encasillar” a nadie, preguntar por preferencias a la hora de desarrollar la app (Saber si alguien prefiere desarrollar el código, las interfaces...)

- Falta de experiencia técnica, dificultades al usar frameworks nuevos o configurar bases de datos/servidores: Riesgo alto
 - Mitigación: Capacitación previa, tutorías, apoyo del profesor o uso de plantillas base.
- Pruebas insuficientes, entregar una app con errores o mal rendimiento: Riesgo medio-alto.
 - Mitigación: Planificar pruebas unitarias, de usabilidad y beta testing con usuarios reales.

Riesgos técnicos:

- Errores en la arquitectura o integración: riesgo alto.
 - Mitigación: Diseñar primero el esquema general con el profesorado, validar la arquitectura antes de programar y empezar por una versión mínima funcional (API + 1 entidad CRUD).
- Problemas con el control de versiones: riesgo medio.
 - Mitigación: Uso obligatorio de GitHub, commits frecuentes y revisión cruzada entre miembros.
- Dificultad para conectar el backend con la app móvil: riesgo medio.
 - Mitigación: Probar la API con Postman antes de desarrollar la interfaz móvil y documentar los endpoints.
- Escalabilidad y rendimiento, si la app crece (más usuarios, más datos), puede volverse lenta o inestable. riesgo bajo
 - Mitigación: Diseñar una base de datos optimizada y planificar desde el inicio una arquitectura modular y escalable.
- Implementación de la gamificación, dificultad técnica en la lógica de puntos, logros, niveles o recompensas. riesgo alto.
 - Mitigación: Diseñar bien el sistema antes de programarlo, con diagramas de flujo y prototipos funcionales.

4. Arquitectura de alto nivel

Aquí se mostrará la arquitectura de nuestra aplicación.

Donde tendremos los siguientes componentes:

Usuario / Cliente: El usuario podrá entrar desde una cuenta en google, con su numero de teléfono, o con un nombre de usuario (el cual luego le pedirá la confirmación del correo electrónico)

Front-End: En este apartado se utilizará Jetpack Compose junto con Android studio.

(Actualizado Hito 4)

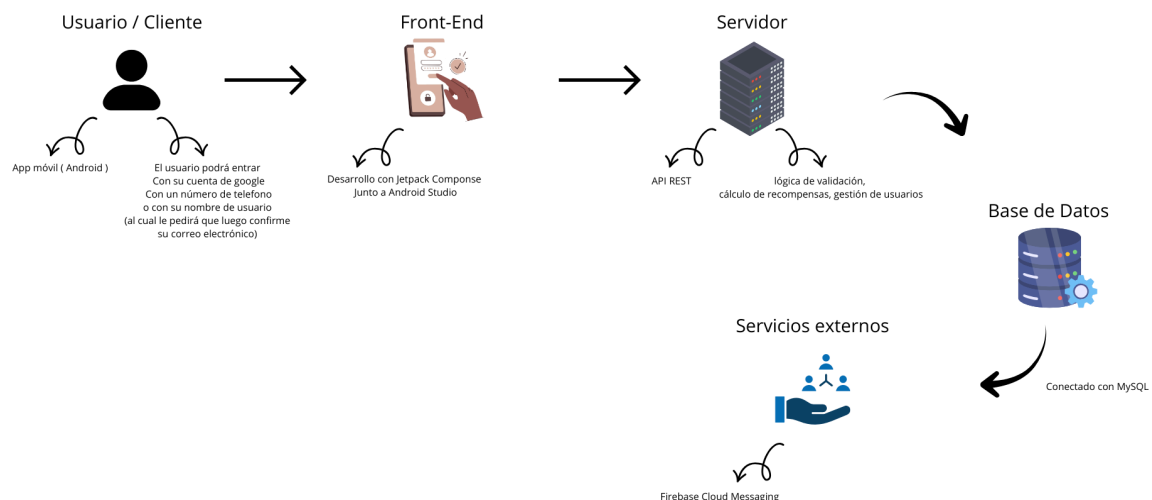
Base de datos:

Se utilizará enfoque híbrido. Para la persistencia local de la app utilizaremos Room Database (SQLite).

Para la base de datos estaremos utilizando Firebase Firestore.

Servicios externos :

Adicionalmente, emplearemos SharedPreferences para almacenar a través de un archivo local en los Assets.

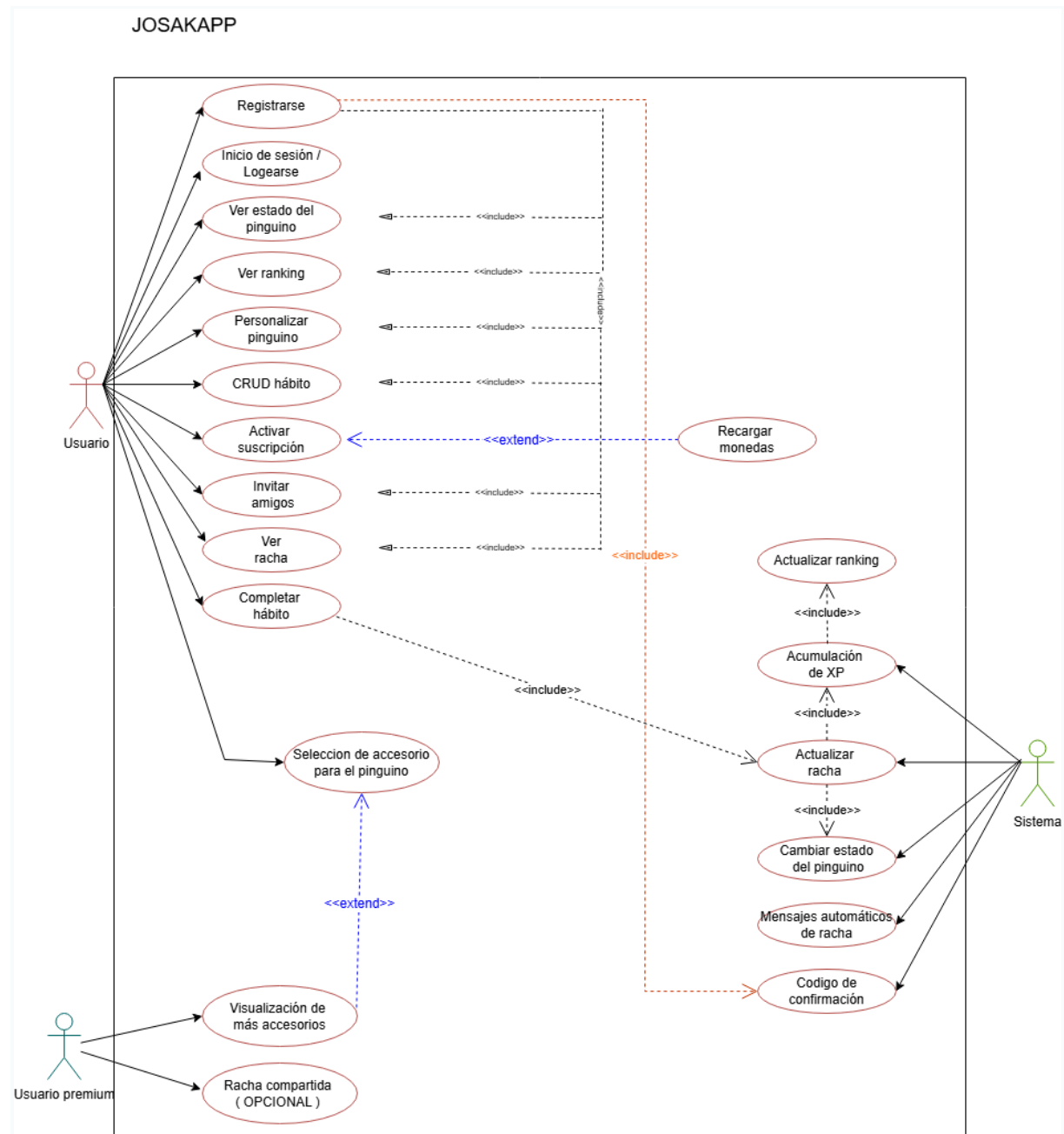


5.Casos de uso UML

En este diseño, se representan funcionalidades principales de JOSA KAPP y cómo interactúan con ellas los distintos tipos de usuarios: el usuario normal, usuario premium y el sistema automatizado.

Permite visualizar de forma clara las acciones que puede realizar cada actor, desde el registro y la gestión de hábitos, hasta la personalización del pinguino y el seguimiento de rachas.

Además, destaca las funciones exclusivas para usuarios premium (que sería tener los accesorios con más anticipación si es que lo quiere o tener una racha compartida con sus amigos (esta parte es opcional)).

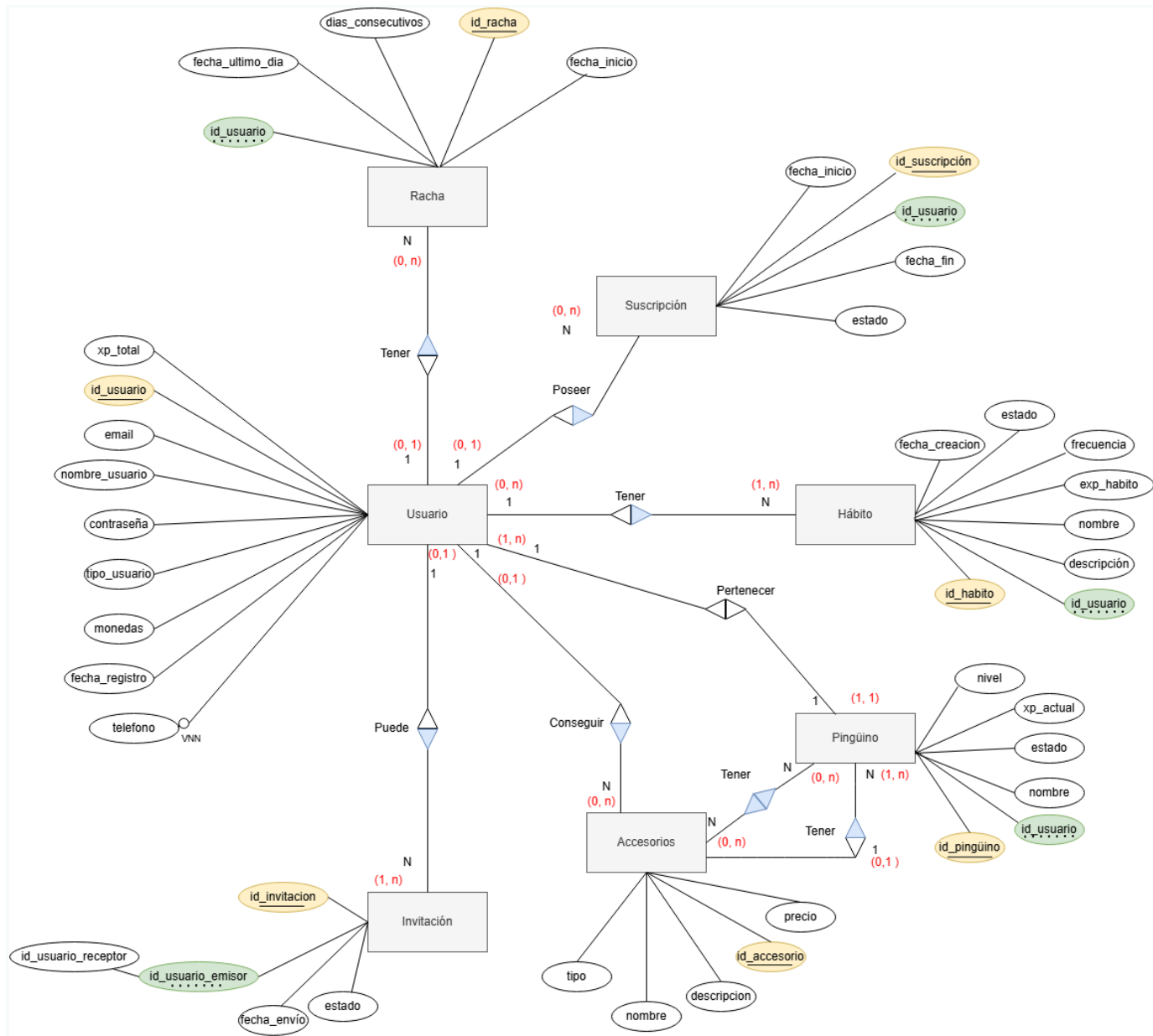


6. Diagrama de Entidad Relación

6.1. Entidades principales:

Las entidades principales son:

Usuario con racha, y usuario con hábito, ya que de esto se trata nuestra aplicación.



Se han implementado una relación de Muchos a Muchos entre pinguino y accesorios utilizando la tabla cruzada PinguinoAccesoriosCrossRef, para poder equipar y desequipar en la base de datos local.

Entidades pendientes: Invitacion, suscripcion y racha aparecen en el modelado inicial pero no se materializan como tablas principales activas de la base de datos.

7. Plan de pruebas inicial

Este apartado define las pruebas que se realizarán sobre las funcionalidades básicas del MVP (Producto Mínimo Viable) de la aplicación.

El objetivo es comprobar que las principales operaciones (login, gestión de usuario, registro de hábitos y validaciones) funcionan correctamente antes de añadir módulos avanzados de gamificación o componentes sociales.

Nº	Funcionalidad	Descripción de la prueba	Acción a realizar	Resultado esperado
1	Registro de usuario (DAO + validaciones)	Comprobar que el sistema permite crear un nuevo usuario correctamente.	Introducir datos válidos en el formulario de registro y pulsar "Crear cuenta".	El usuario se registra y se guarda en la base de datos; redirección a la pantalla principal.
2	Registro con campos vacíos	Verificar la validación del formulario.	Dejar uno o más campos en blanco y pulsar "Crear cuenta".	Aparece un mensaje de error indicando los campos obligatorios.
3	Login correcto	Validar autenticación con credenciales existentes.	Introducir email y contraseña válidos.	Acceso correcto al perfil del usuario.
4	Login con credenciales incorrectas	Comprobar respuesta ante datos erróneos.	Introducir email o contraseña no registrados.	Mensaje de error ("Credenciales incorrectas") y no acceso al sistema.
5	Creación de hábito	Validar inserción	Introducir un	El hábito

	diario	de nuevos hábitos en el registro.	nuevo hábito ("Beber agua", por ejemplo) y guardar.	aparece en la lista diaria y se almacena en la base de datos.
6	Actualización de hábito	Verificar que se pueden editar hábitos existentes.	Modificar el título o la frecuencia de un hábito.	Cambios guardados correctamente y reflejados en la interfaz.
7	Eliminación de hábito	Confirmar que se puede borrar un hábito.	Seleccionar un hábito y pulsar "Eliminar".	El hábito desaparece de la lista y se elimina de la base de datos.
8	Validación de formulario de hábitos	Comprobar que no se puede guardar un hábito vacío o sin nombre.	Dejar el campo de nombre vacío y guardar.	Aparece un mensaje de error ("El nombre del hábito es obligatorio").
9	Notificación diaria	Verificar que el sistema envía recordatorios.	Configurar un hábito con recordatorio diario y esperar la hora programada.	El dispositivo muestra una notificación push con el mensaje configurado.
10	Navegación entre pantallas	Evaluar la fluidez y coherencia del flujo de usuario.	Navegar entre pantalla de inicio, hábitos y perfil.	No hay errores ni cierres inesperados; navegación fluida.
11	Persistencia de datos	Asegurar que los datos permanecen tras cerrar sesión.	Crear hábitos, cerrar la app y volver a iniciar sesión.	Los hábitos creados se mantienen visibles y sincronizados.
12	Desempeño básico	Comprobar tiempos de carga aceptables.	Iniciar sesión y cargar la lista de hábitos.	La app responde en menos de 2 segundos en la mayoría de operaciones.

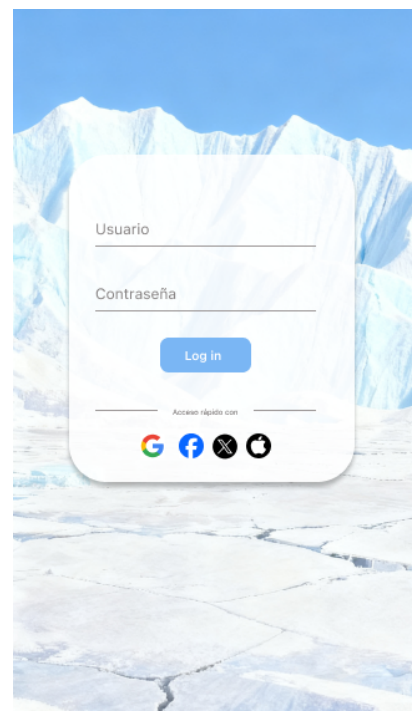
8.Mockups navegables de todas las pantallas

Enlace de figma

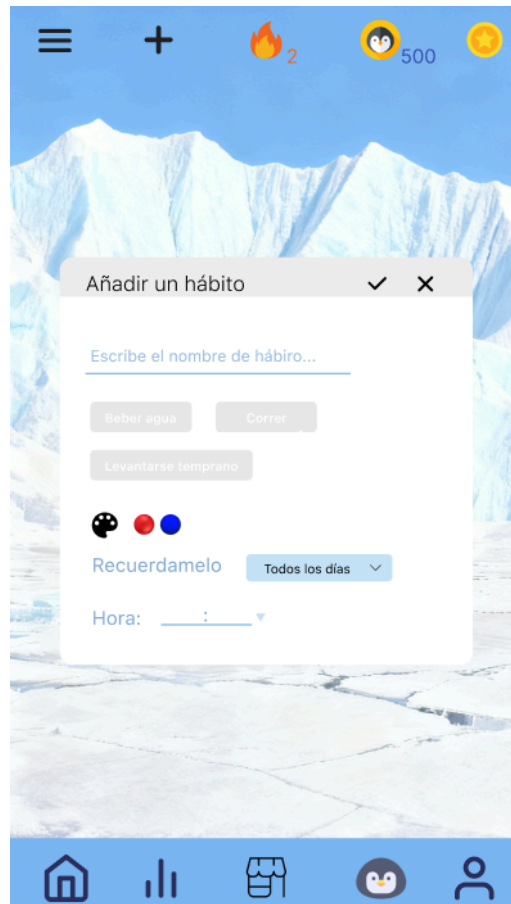
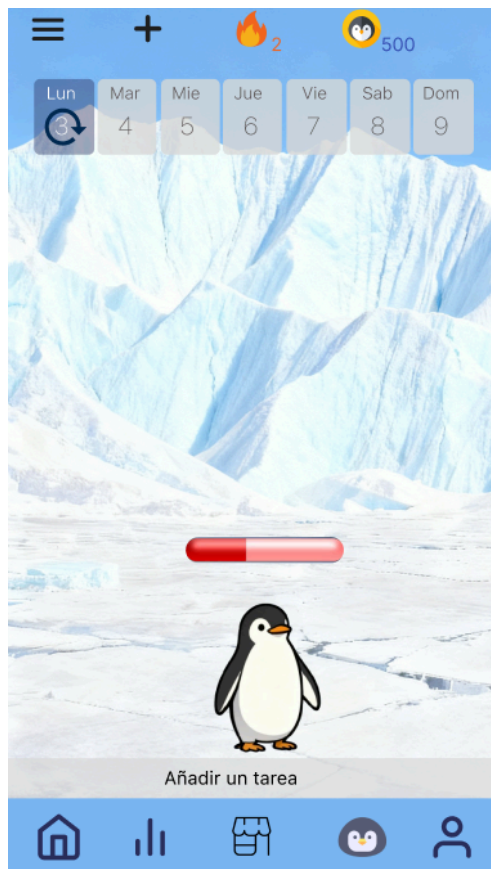
[figma.com/design/WNFdA7UtEsM04cN4LTevMj/josak-app?t=0MCixYOgaCZfx0Ak-0](https://www.figma.com/design/WNFdA7UtEsM04cN4LTevMj/josak-app?t=0MCixYOgaCZfx0Ak-0)

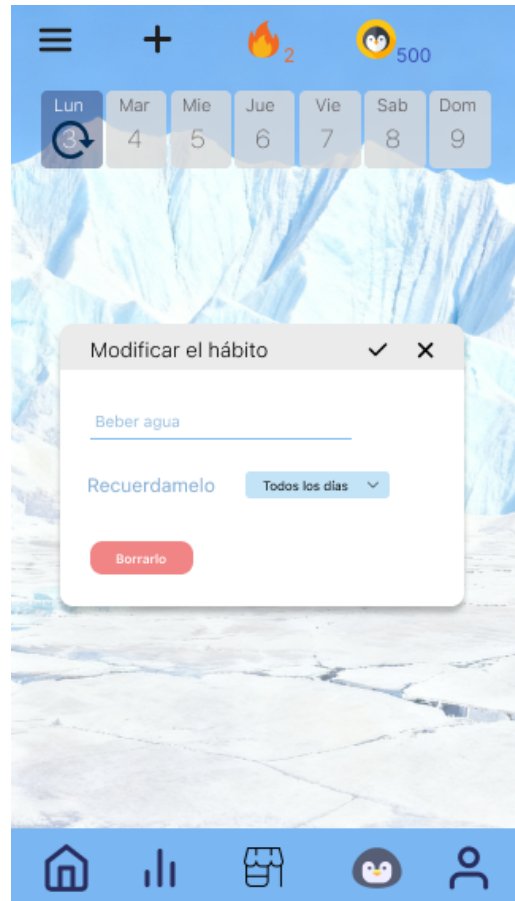
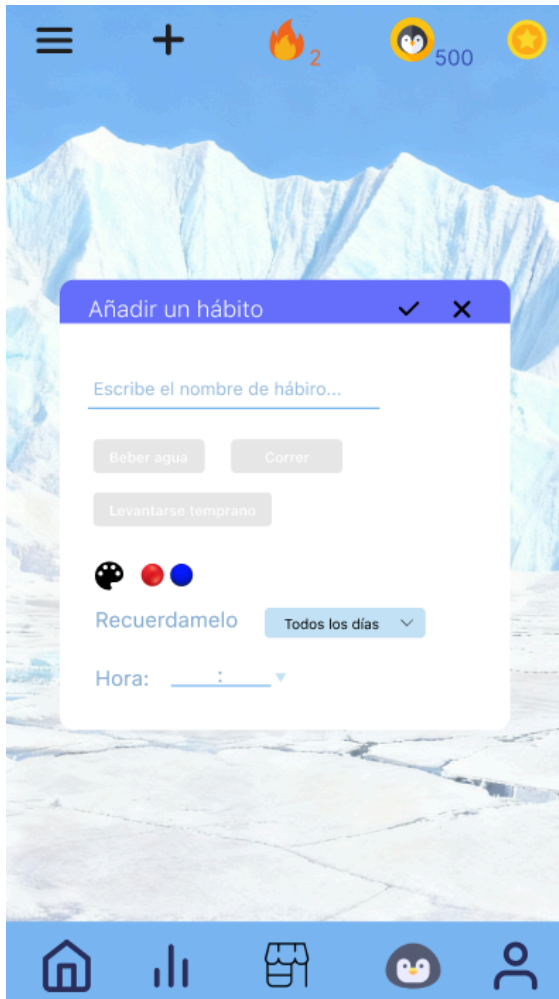
<https://www.figma.com/proto/WNFdA7UtEsM04cN4LTevMj/josak-app?node-id=1-4&t=u5XhvAWX6AJzZcNn-0&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=1%3A4>

Pantalla de Login

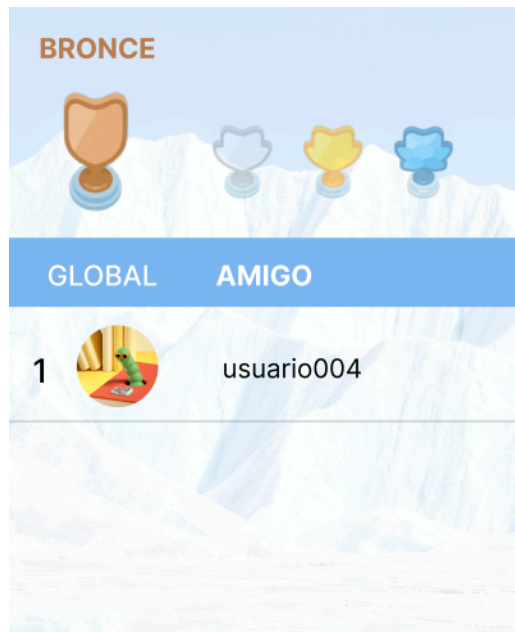
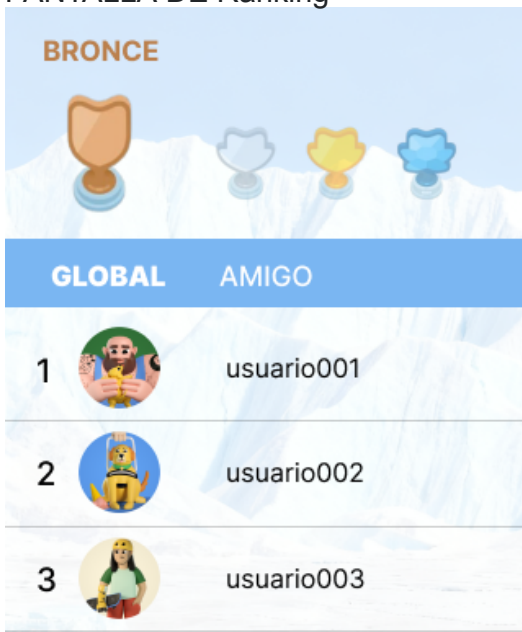


PANTALLA DE INICIO

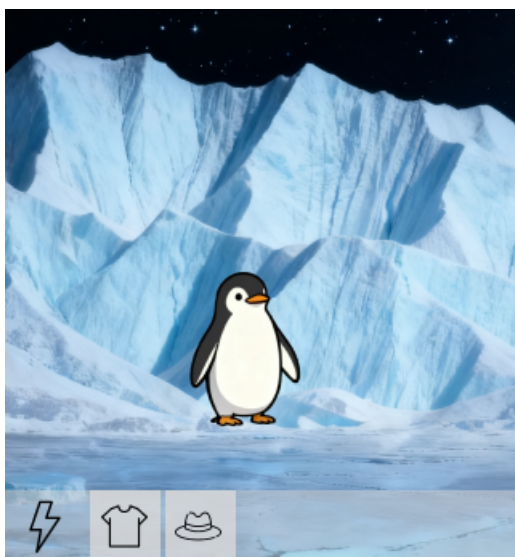
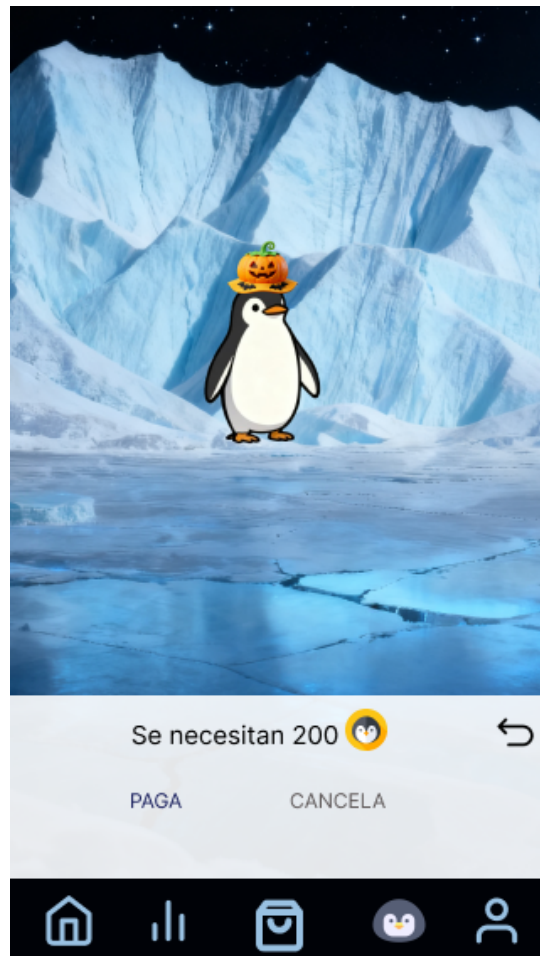
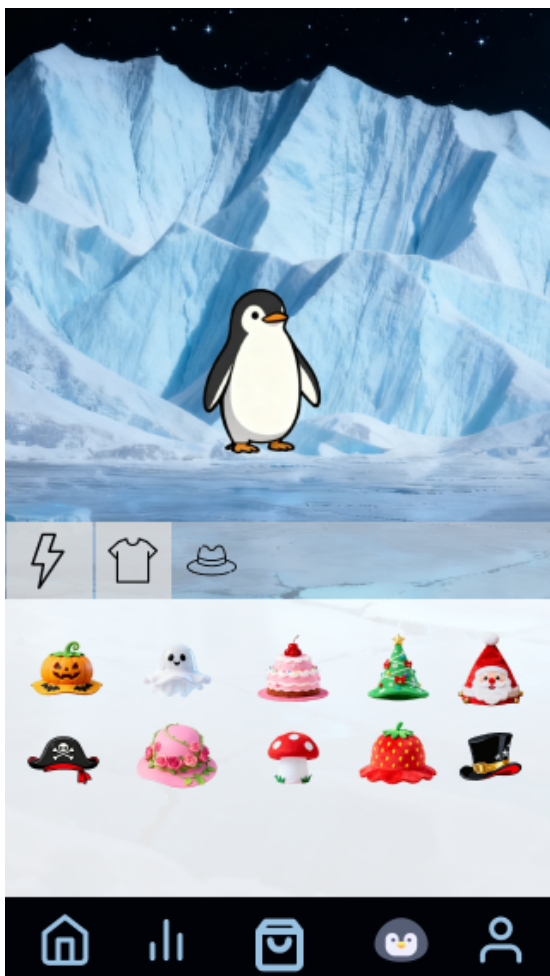




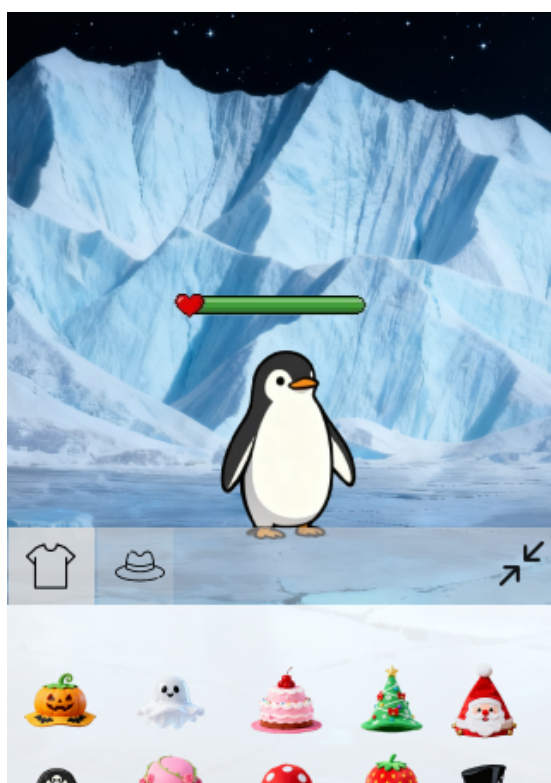
PANTALLA DE Ranking

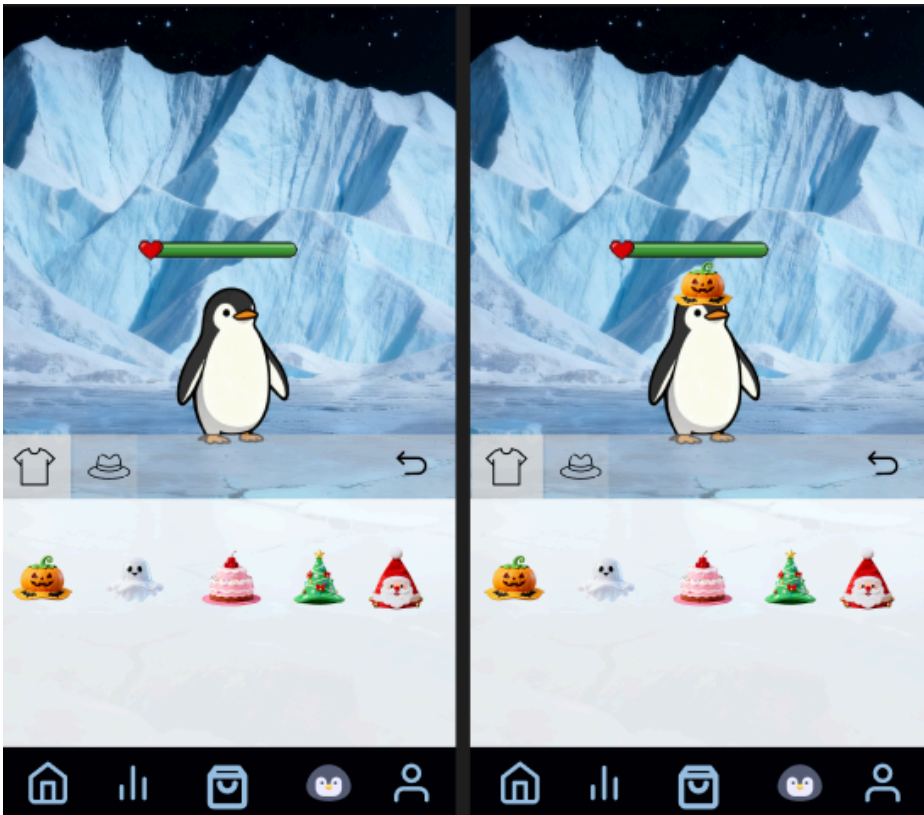
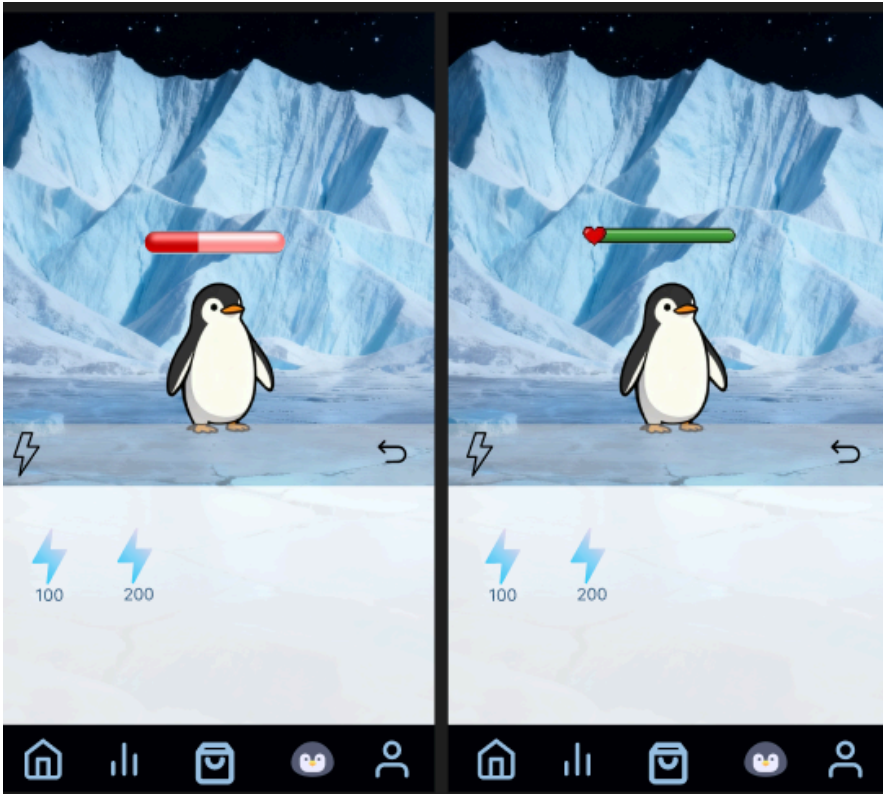


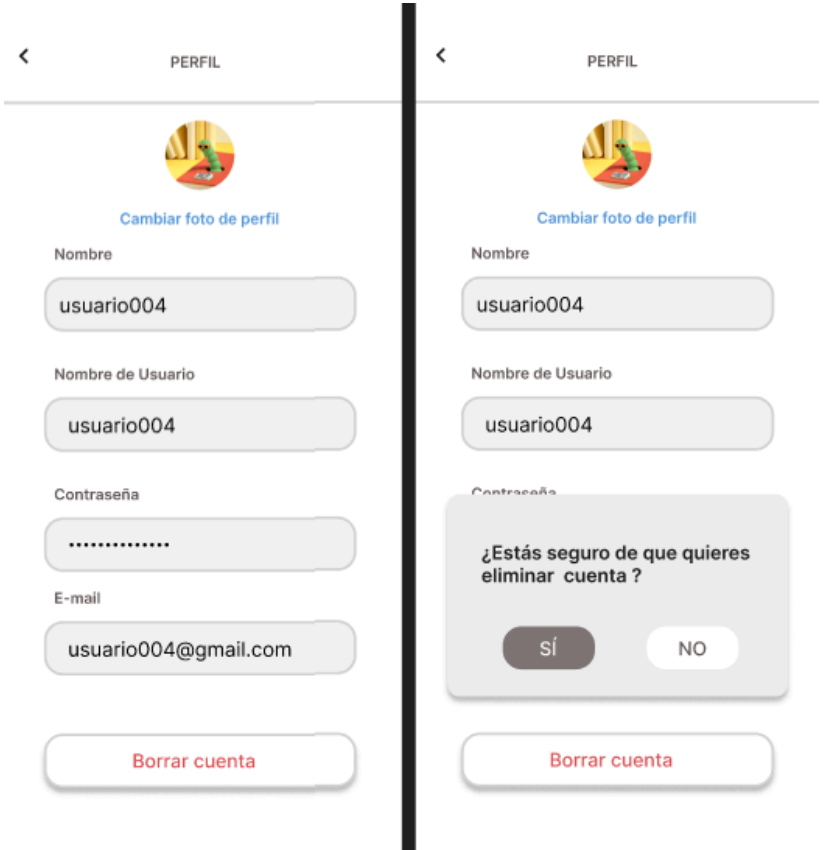
PANTALLA DE Tienta



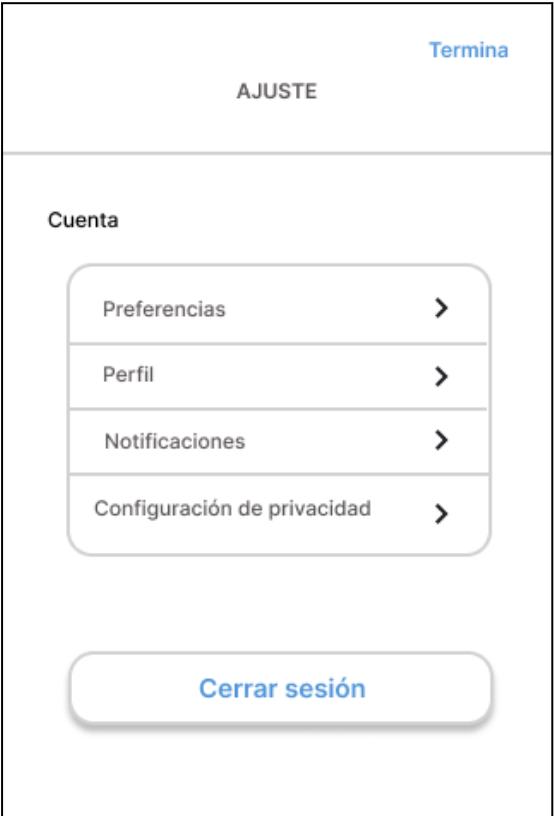
PANTALLA DE Pingüino

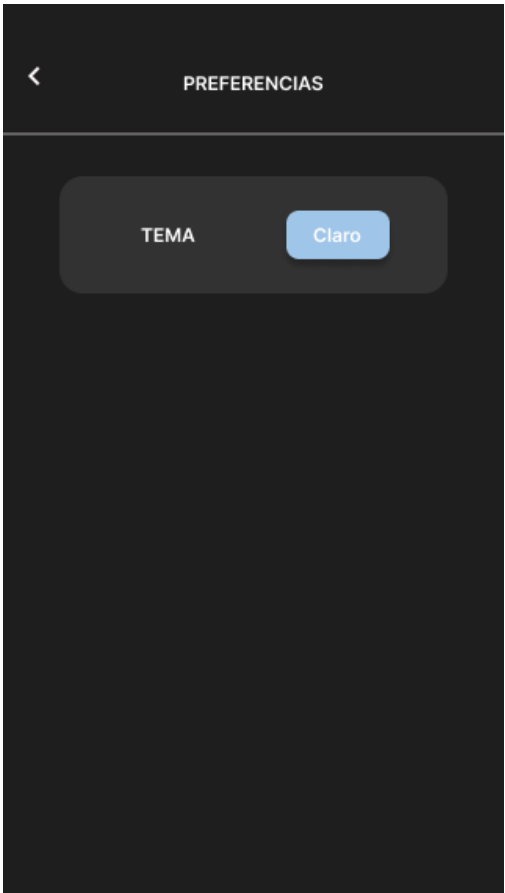
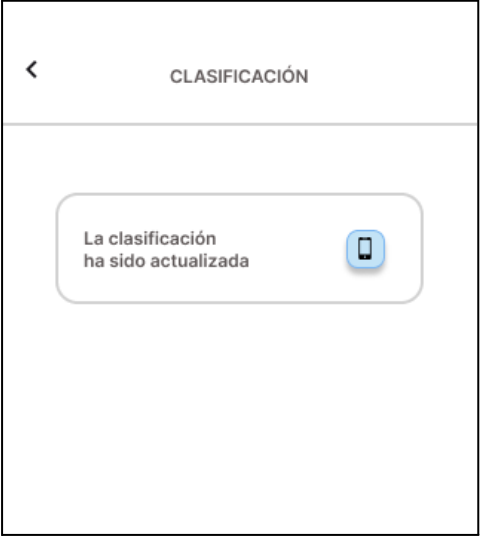






PANTALLA DE Ajuste







PRIVACIDAD

Usted confía en la manera en que tratamos su información al utilizar los servicios de Josak App.

Somos plenamente conscientes de que esta responsabilidad es muy importante, por lo que nos comprometemos a proteger su información y a brindarle control sobre ella.

Esta Política de Privacidad tiene como objetivo ayudarle a entender qué información recopilamos, por qué la recopilamos y cómo puede actualizar, gestionar, exportar y eliminar su información.

Si las leyes de protección de datos de la Unión Europea o del Reino Unido se aplican al tratamiento de su información, puede consultar la sección "Requisitos en Europa" más abajo para obtener información detallada sobre sus derechos y sobre cómo Google cumple con dichas leyes.

HITO 3

1. Resumen del MVP-1 y alcance implementado

En este Hito 3 se ha implementado el flujo completo del usuario: registro e inicio de sesión.

Se ha desarrollado el panel de hábitos con funciones CRUD (creación, modificación y eliminación de hábitos), lista de hábitos, calendario (con los días de constancia), tienda de monedas y visualización de un pingüino.

En el panel de ranking tenemos una lista con el ranking de amigos y el ranking global.

En el panel de Tienda se muestran los puntos del usuario y los artículos disponibles.

En el panel de Perfil se ha desarrollado la edición del perfil, las preferencias de tema (oscuro/claro), las notificaciones (recordatorios, amigos, clasificación y anuncios) y la configuración de privacidad.

1.1. Cambios que se hizo en el diseño E-R en el Hito 2:

> Primero se realizaron nuevas relaciones :

Habito con usuario (1: N) : ya que un usuario puede tener muchos hábitos pero un hábito puede pertenecer solo a un usuario (por el momento se queda así).

> Relación de usuario con pingüino (1: 1):

Aquí lo que se pensó es que por el momento allá un pingüino para todo. Ya que no se sabe si se llegará a lo del hábito compartido.

> Relaciones entre pingüino y accesorio 2:

Se penso así por que existe diferencia entre lo que lleva el pingüino y todo lo que tiene guardado, items que compro o que ganó cuando completo la racha.

La primera de N:N : Para que pueda ver todo el armario, lo que el usuario ha desbloqueado con puntos.

La segunda de 1:N : El accesorio que lleva el pingüino.

El pingüino solo puede llevar puesto un accesorio en especifico en un momento.

2. Funcionalidades del MVP-1 (Casos de Uso Implementados)

Caso de Uso	Implementado	En Progreso	Pendiente
CU1 - Registrarse	X		
CU2 - Iniciar Sesión	X		
CU3 – CRUD hábitos	X		
CU4 – Ver ranking		La parte de amigos sí, pero nos faltaría la parte global	
CU5-Sistema de recompensas con monedas		Implenta la pantalla de moneda,pero el función está en progreso	
CU6-personalizar al pingüino			X
CU7-Notificaciones push para recordatorios de hábitos			No implementado (la estructura quedo implementada pero no se integro)
CU8 - Invitar amigos			X
CU9 - Activar suscripción			No implementado (la estructura quedo implementada pero no se integro)
CU10 - Ver racha	X		
CU11 - Completar el hábito	X		

CU12 - Selección de accesorio para el pinguino			X
CU13 - Visualización de más accesorios (Usuario Premium)			X
CU14 -Ver estado del pingüino		Implementa el mostrar pingüino pero falta implementa mostrar el estado	

Recorrido de flujo de Principio a Fin:

Ejemplo login: Iniciar sesión -> Completaría los datos, que serían el nombre y la contraseña -> se guarda en la base de datos que por el momento estamos utilizando Firebase

Ejemplo completar hábitos: Crear hábito -> El usuario ingresaría el nombre del hábito y a que hora quiere que se le recuerde, también hay iconos y colores que podrá cambiar. -> luego cada vez que lo complete se le dará 10 XP y se actualizará la racha por día.

3. Arquitectura Implementada y Entorno de Ejecución

Tecnologías utilizadas:

- **Frontend:** En el frontend hemos utilizado Jetpack Compose junto a Android Studio.
Lenguaje: Kotlin
- **Backend y Base de Datos:** La tecnología que se utilizaron fueron Google Firebase. (NoSQL) en lugar de MySQL.
Por el momento optamos por Firestore por su capacidad de sincronización y la integración con Android que la hace fácil.
- **Servicios externos:** Firebase Auth (para el registro y para el login del usuario)

Entornos de Ejecución:

La aplicación se ejecuta en entorno local mediante el emulador de Android Studio.

La persistencia de datos reside en la nube (Cloud) de Google Firebase, lo que permite que los datos no se pierdan al cerrar la aplicación

4.Base de datos y operaciones CRUD implementadas

Se han creado las tablas haibito, user, amigo y HabitoRegistro en SQL. Se ha implementado el CRUD completo de usuario,habito,amigo a través de la capa DAO, utilizando consultas parametrizadas para evitar inyección SQL y Repository.

5. Plan de pruebas ejecutado (con resultado)

ID	Caso	Resultado esperado	Resultado obtenido	Estado
T1	Registro de nuevo usuario	Crear documento en Firebase Auth y Firestore	Usuario creado correctamente	Se verifica en la consola de Firebase
T2	Login con datos erróneos	Mostrar mensajes de error y denegar acceso	El sistema avisa si hay algo incorrecto	Validación nativa de Firebase

6. Repositorio y control de versiones

<https://github.com/saletdiaz/JosakApp>